

Pregunta 1: ¿Cuál fue la diferencia exacta de tiempo (en ms) entre el procesamiento normal y el procesamiento con el bloqueo de 1ms?

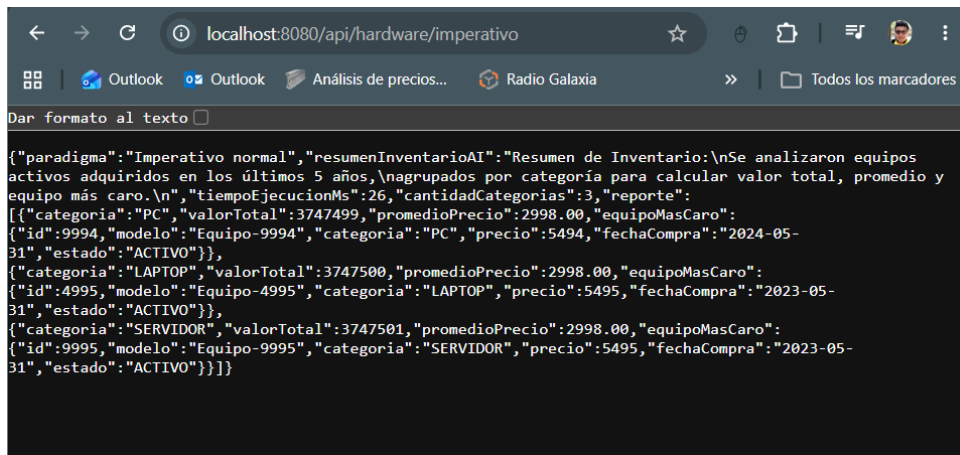
Durante la experimentación se obtuvieron los siguientes resultados:

- Paradigma Imperativo normal: 26 ms
- Paradigma Imperativo con bloqueo: 7822 ms
- Paradigma Funcional normal: 37 ms
- Paradigma Funcional con bloqueo: 7828 ms

Por lo tanto, la diferencia exacta fue:

- Imperativo: $7822 \text{ ms} - 26 \text{ ms} = \mathbf{7796 \text{ ms}}$
- Funcional: $7828 \text{ ms} - 37 \text{ ms} = \mathbf{7791 \text{ ms}}$

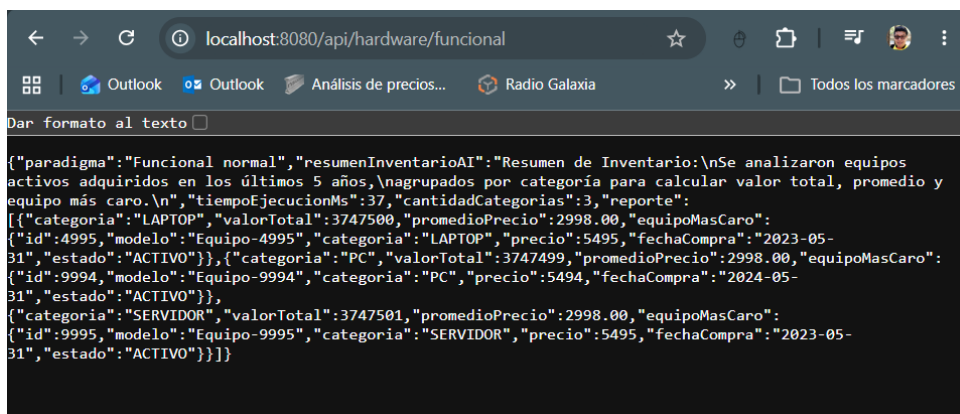
Esto demuestra que la introducción del bloqueo mediante `Thread.sleep(1)` incrementó significativamente el tiempo de ejecución en ambos enfoques.



The screenshot shows a web browser window with the address bar displaying `localhost:8080/api/hardware/imperativo`. The browser's taskbar at the top includes icons for Outlook, 'Análisis de precios...', and Radio Galaxia. The main content area displays a JSON response from the API. The JSON object contains a 'paradigma' field set to 'Imperativo normal' and a 'resumenInventarioAI' field with a detailed report. The report includes a summary of equipment analysis and a list of three categories: PC, LAPTOP, and SERVIDOR, each with associated values for total value, average price, and the most expensive equipment.

```
{
  "paradigma": "Imperativo normal",
  "resumenInventarioAI": "Resumen de Inventario:\nSe analizaron equipos activos adquiridos en los \u00faltimos 5 a\u00f1os,\nagrupados por categor\u00eda para calcular valor total, promedio y equipo m\u00e1s caro.\n",
  "tiempoEjecucionMs": 26,
  "cantidadCategorias": 3,
  "reporte": [
    {
      "categoria": "PC",
      "valorTotal": 3747499,
      "promedioPrecio": 2998.00,
      "equipoMasCaro": {
        "id": 9994,
        "modelo": "Equipo-9994",
        "categoria": "PC",
        "precio": 5494,
        "fechaCompra": "2024-05-31",
        "estado": "ACTIVO"
      }
    },
    {
      "categoria": "LAPTOP",
      "valorTotal": 3747500,
      "promedioPrecio": 2998.00,
      "equipoMasCaro": {
        "id": 4995,
        "modelo": "Equipo-4995",
        "categoria": "LAPTOP",
        "precio": 5495,
        "fechaCompra": "2023-05-31",
        "estado": "ACTIVO"
      }
    },
    {
      "categoria": "SERVIDOR",
      "valorTotal": 3747501,
      "promedioPrecio": 2998.00,
      "equipoMasCaro": {
        "id": 9995,
        "modelo": "Equipo-9995",
        "categoria": "SERVIDOR",
        "precio": 5495,
        "fechaCompra": "2023-05-31",
        "estado": "ACTIVO"
      }
    }
  ]
}
```

Figura 1. Resultado del paradigma imperativo normal (26 ms).



The screenshot shows a web browser window with the address bar displaying `localhost:8080/api/hardware/funcional`. The browser's taskbar at the top includes icons for Outlook, 'An\u00e1lisis de precios...', and Radio Galaxia. The main content area displays a JSON response from the API. The JSON object contains a 'paradigma' field set to 'Funcional normal' and a 'resumenInventarioAI' field with a detailed report. The report includes a summary of equipment analysis and a list of three categories: LAPTOP, PC, and SERVIDOR, each with associated values for total value, average price, and the most expensive equipment.

```
{
  "paradigma": "Funcional normal",
  "resumenInventarioAI": "Resumen de Inventario:\nSe analizaron equipos activos adquiridos en los \u00faltimos 5 a\u00f1os,\nagrupados por categor\u00eda para calcular valor total, promedio y equipo m\u00e1s caro.\n",
  "tiempoEjecucionMs": 37,
  "cantidadCategorias": 3,
  "reporte": [
    {
      "categoria": "LAPTOP",
      "valorTotal": 3747500,
      "promedioPrecio": 2998.00,
      "equipoMasCaro": {
        "id": 4995,
        "modelo": "Equipo-4995",
        "categoria": "LAPTOP",
        "precio": 5495,
        "fechaCompra": "2023-05-31",
        "estado": "ACTIVO"
      }
    },
    {
      "categoria": "PC",
      "valorTotal": 3747499,
      "promedioPrecio": 2998.00,
      "equipoMasCaro": {
        "id": 9994,
        "modelo": "Equipo-9994",
        "categoria": "PC",
        "precio": 5494,
        "fechaCompra": "2024-05-31",
        "estado": "ACTIVO"
      }
    },
    {
      "categoria": "SERVIDOR",
      "valorTotal": 3747501,
      "promedioPrecio": 2998.00,
      "equipoMasCaro": {
        "id": 9995,
        "modelo": "Equipo-9995",
        "categoria": "SERVIDOR",
        "precio": 5495,
        "fechaCompra": "2023-05-31",
        "estado": "ACTIVO"
      }
    }
  ]
}
```

Figura 2. Resultado del paradigma funcional normal (37 ms).

Pregunta 2: Si tu servidor tiene un límite de 200 hilos (configuración por defecto de Tomcat), ¿cuántas solicitudes concurrentes podrían procesarse por segundo si cada una tarda lo que marcó tu experimento?

Para estimar las solicitudes por segundo se utiliza la fórmula:

Solicitudes por segundo $\approx$ Número de hilos / Tiempo por solicitud (en segundos)

Tomando los tiempos con bloqueo:

- Imperativo: 7822 ms = 7,822 s
- Funcional: 7828 ms = 7,828 s

Cálculo:

- Imperativo: $200 / 7,822 \approx 25,57$ solicitudes por segundo
- Funcional: $200 / 7,828 \approx 25,55$ solicitudes por segundo

Por lo tanto, con un límite de 200 hilos, el servidor podría procesar aproximadamente **25 solicitudes por segundo** cuando cada solicitud permanece bloqueada durante el tiempo observado en el experimento.



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/api/hardware/imperativo-bloqueante'. The browser tabs include Outlook, Outlook, Análisis de precios..., and Radio Galaxia. The main content area displays a JSON response from the API. The JSON object contains a 'paradigma' field with the value 'Imperativo con bloqueo Thread.sleep(1)', a 'resumenInventarioAI' field with a detailed summary of inventory analysis, and a 'reporte' field containing an array of equipment data. The equipment data includes fields for 'id', 'modelo', 'categoria', 'precio', and 'fechaCompra'. The JSON is formatted with line breaks for readability.

Figura 3. Resultado del paradigma imperativo con bloqueo (7822 ms).



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/api/hardware/funcional-bloqueante'. The browser tabs include Outlook, Outlook, Análisis de precios..., and Radio Galaxia. The main content area displays a JSON response from the API. The JSON object contains a 'paradigma' field with the value 'Funcional con bloqueo Thread.sleep(1)', a 'resumenInventarioAI' field with a detailed summary of inventory analysis, and a 'reporte' field containing an array of equipment data. The equipment data includes fields for 'id', 'modelo', 'categoria', 'precio', and 'fechaCompra'. The JSON is formatted with line breaks for readability.

Figura 4. Resultado del paradigma funcional con bloqueo (7828 ms).

Pregunta 3: ¿Observaste alguna diferencia significativa de rendimiento entre el enfoque Imperativo y el Funcional al introducir el bloqueo? Justifica por qué.

No se observó una diferencia significativa de rendimiento entre ambos enfoques al introducir el bloqueo. El paradigma imperativo registró 7822 ms y el funcional 7828 ms, existiendo únicamente una diferencia de 6 ms.

Esto ocurre porque el costo principal ya no proviene del procesamiento del algoritmo, sino del tiempo de espera generado por `Thread.sleep(1)`. Cuando una operación bloqueante domina el tiempo total de ejecución, las diferencias entre utilizar ciclos tradicionales o Streams se vuelven prácticamente irrelevantes. En consecuencia, ambos paradigmas presentan un comportamiento muy similar bajo condiciones de bloqueo.

Pregunta 4: En un entorno real, ¿qué alternativas existen en Spring Boot para evitar que un hilo se quede bloqueado esperando una respuesta de un servicio externo?

En Spring Boot existen varias alternativas para evitar que un hilo permanezca bloqueado mientras espera la respuesta de un servicio externo:

1. **Spring WebFlux**, que utiliza programación reactiva y procesamiento no bloqueante.
2. **WebClient**, que permite realizar llamadas HTTP asíncronas y reactivas.
3. **@Async**, para ejecutar tareas en segundo plano utilizando pools de hilos.
4. **CompletableFuture**, que facilita la ejecución asíncrona sin bloquear el hilo principal.
5. **RabbitMQ o Kafka**, que permiten procesar tareas mediante mensajería asíncrona y desacoplada.

Estas alternativas mejoran la escalabilidad y el rendimiento de aplicaciones empresariales que dependen de servicios externos o procesos de larga duración.